Predicting CYP Inhibition: Method Report

OpenADMET CYP Inhibition Blind Challenge (GiantWizard)
Jeffery Chen
November 2026

1. Overview

My entry (GiantWizard) covers both tracks of the OpenADMET CYP Inhibition Blind Challenge: direct inhibition regression on CYP1A2, CYP2C9, CYP2D6, and CYP3A4, plus time-dependent inhibition (TDI) classification on CYP2D6 and CYP3A4. I only used public data. The code and scripts to run everything are up at `github.com/GiantWizard/openadmet-cyp-challenge`.

The regression leaderboard uses macro soft-threshold relative absolute error (MA-ST-RAE). If a prediction lands inside a compound's credible interval, the error is 0. If it lands outside, you get charged for how far it missed the closer edge of the interval. Weak compounds below $\text{pIC}_{50} = 4$ (below the lowest dose tested) have credible intervals about ten times wider than active hits.

I didn't build a custom graph network for this challenge. Every model in my repo is a stock Chemprop, CheMeleon, TabPFN, or LightGBM model. Most of my time went into figuring out how the assay setup and the ST-RAE metric mess with standard models. Four things made the biggest difference:

First, the training CSV has blank cells wherever a molecule failed the initial single-dose screen and got skipped for 12-point dose-response curves. The 750 blind test molecules, on the other hand, were run through dose-response across all four enzymes. On top of that, the organizers built the blind set by buying 10 Enamine catalog analogs for each of the top 25 hits from CYP1A2, CYP2C9, and CYP3A4 while CYP2D6 was still being set up on the Echo-MS machine. So for CYP2D6, most of the 750 test molecules are unexpanded non-inhibitors. Pulling blank primary-screen rows into the training set as weak examples ($\text{pIC}_{50} = 3.5$) jumped my rank from 55 to 43 in one upload, and to 41 with a follow-up weight change.

Second, since the blind set is made of close analogs to training hits, its median Tanimoto similarity to the training set is 0.578. My scaffold-split holdout is at 0.435. Splitting validation folds by strict chemical clusters makes local CV way harder than the actual test set.

Third, training a model directly on the clipped interval loss does the exact same thing as shifting its predictions with a linear transform after training. Each one helps by itself, and using both together cancels out the gain.

Fourth, my early TabPFN scripts shrank the 605 input features down to 200 PCA components so the table would fit on a CPU. Dropping PCA and running all 605 columns on a GPU beat six weeks of blending and calibration tweaks. It was also the only change that improved all five leaderboard metrics at the same time.

On 29 October, my standalone five-arm stack (v46) scored 0.4472. Averaging my predictions with Scigantic's open-source R25 file dropped the score to 0.4335 that same day. From there, widening the spread of each column without moving its mean or compound ranking brought my final upload (v56) to 0.4192.

2. Main Findings

2.1 Models trained only on follow-up hits never see weak compounds

In the training CSV, a lot of pIC_{50} cells are blank. On 8 October, the organizers mentioned on Discord that a blank cell means the molecule didn't inhibit enough in the single-concentration screen (+NADPH pre-incubation) to pass the log-fold change and Benjamini-Hochberg FDR cutoffs for 12-point dose-response testing. For the 750 blind test compounds, though, they ran full curves on all four isoforms.

This hits CYP2D6 twice. For one, CYP2D6 was run on a SCIEX Echo-MS system (dextromethorphan depletion) while the other three isoforms used Vivid fluorescence kits. And since the 750 test compounds were picked by expanding hits on 1A2, 2C9, and 3A4 before the 2D6 Echo-MS setup was finished, almost the whole test set is inactive on CYP2D6.

If you look at the 1,493 labelled CYP2D6 rows in the training set, every single one has a primary-screen response of -0.59 or stronger ($\log_2$ fold change). Out of the 2,882 blank CYP2D6 rows that have primary-screen numbers, only 5 reach -0.73. So if you only train on rows with pIC_{50} numbers, the model has zero inactive examples. CYP3A4 and CYP2C9 don't work like this, since their blank cells still have plenty of strong primary-screen hits.

To fix CYP2D6, I grabbed blank rows with a weak primary-screen reading (weaker than -0.72, which is the 95th percentile of the labelled rows) and added them to the training data at $pIC_{50} = 3.5$, up to one blank row per labelled row. I only used the screen reading to pick which blank rows to include; the model doesn't get the screen reading as a feature since the test CSV doesn't have it. I trained a TabPFN regressor on this expanded table using the organizers' `surrogate_adme` embedding.

My locked holdout couldn't test this properly since the holdout only has active hits, and the new arm actually scored worse there on CYP2D6 (0.7966 to 0.8156 ST-RAE). So I checked two other numbers on my folds before uploading. First, the chance that a labelled hit gets predicted higher than a blank row went up from 0.752 to 0.829 on dev and from 0.741 to 0.824 on the holdout. Second, Spearman rank correlation on the labelled hits barely moved (0.4460 vs. 0.4499 on dev; 0.5563 vs. 0.5444 on holdout). On the 750 blind test molecules, the share of CYP2D6 predictions below 3.8 jumped from 1% to 36%.

When I uploaded a file with only the CYP2D6 column replaced (v26), all five leaderboard metrics improved at once: macro ST-RAE went from 0.4938 to 0.4806, Spearman from 0.7234 to 0.7382, R^2 from 0.5660 to 0.5845, and my rank jumped from 55 to 43. Since I kept the other three columns identical and the macro score is a straight average of the four tabs, CYP2D6 improved by 0.053 on ST-RAE and 0.059 on Spearman. In my next upload (v27), I bumped that model's weight in my CYP2D6 blend from 0.4 to 0.7 (averaging four floor and row-count setups), which got me to 0.4743 and rank 41. Trying the same trick on CYP1A2 did almost nothing, since the base model already separated blanks from hits at 0.86 AUC.

2.2 Dropping PCA before TabPFN

Early on, all of my TabPFN models used PCA to shrink their 605 input columns down to 200 components. I had put that step in when I was running on a CPU and the full table wouldn't fit in memory. Once I moved to a GPU, I took PCA out and gave TabPFN all 605 raw columns. That one change added +0.0125 to my macro dev Spearman, bigger than anything else I tried over six

weeks.

On the live board, that upload pushed all five metrics in the right direction: ST-RAE dropped from 0.5433 to 0.5333, MAE went from 0.7181 to 0.7094, R^2 from 0.5211 to 0.5317, Spearman from 0.6975 to 0.7041, and Kendall from 0.5224 to 0.5269. Every calibration change I had done before that just traded one metric for another.

The CPU memory flags only cost 0.0023 on their own. The problem was that PCA picks 200 directions to explain variance in the chemical descriptors, throwing away smaller directions that matter for CYP inhibition.

2.3 Why cluster-disjoint CV is too harsh for this test set

In most molecular ML competitions, you want to split cross-validation folds by chemical clusters so similar series don't leak across folds. One team ahead of me on the board even pointed to cluster-disjoint splitting as the main reason their local CV matched the leaderboard.

When I checked the Tanimoto similarities here, they told the opposite story. The organizers built the 750-compound blind set by taking 75 potent hits and buying their 10 closest Tanimoto neighbors from Enamine. That puts the blind set at a median Morgan Tanimoto similarity of 0.578 to the training set. My scaffold-disjoint holdout is down at 0.435 (Scigantic measured 0.587 on the blind set vs. 0.425 on a scaffold split and 0.439 on a random split).

Split compared to training set (4,905 rows)	Median Tanimoto	> 0.4	> 0.6
750 Blind Test Compounds	0.578	97.0%	40.0%
726 Scaffold-Disjoint Holdout Compounds	0.435	65.0%	5.5%

My scaffold holdout is already a lot further from the training data than the blind test set is. That's why my holdout ST-RAE sits around 0.60 while the leaderboard scores around 0.51. Making the CV split even stricter with cluster separation just pushes local validation further away from the close-analog setup I'm being scored on.

2.4 Getting real diversity in Chemprop ensembles

I trained four Chemprop D-MPNNs starting from the same public pretrained checkpoint. Between the four runs, I changed the number of auxiliary heads (13 vs. 20), how many seeds I averaged (1 vs. 3), whether ChEMBL rows were mixed in, and whether the TDI-arm pIC_{50} columns were included. On held-out folds, all four models correlated above 0.95 with each other. As soon as the combiner gave weight to one of them, it zeroed out the other three.

Then I trained a fifth model using the exact same script and training table, but started it from a different public checkpoint. Its predictions correlated at 0.70 to 0.90 with the first four. Even though it scored 0.046 worse than my best single net on its own, the combiner gave it a 0.25 weight on three of the four isoforms. Changing seeds and auxiliary heads barely moved the predictions; to get a different view of the data, I had to start from a different pretraining corpus.

2.5 Solving for the blind test distribution from leaderboard scores

Every time you upload a file, the leaderboard gives five numbers per isoform: ST-RAE, MAE, R^2, Spearman, and Kendall. Team `briford` (SuperCowPowers) posted a great write-up showing that

those metrics lock in three unknowns: the blind test set's mean, its standard deviation, and your correlation with truth. Instead of using three uploads to solve the equations, I wrote a solver that gets the moments from a single upload (by adding a simulated ST-RAE equation) and adapted the same idea to the TDI classification board to find the hidden positive rates.

3. Ideas That Didn't Work

I ran a bunch of other experiments on the dev folds that didn't pan out:

Idea	What I ran	What happened
Public potency panels as features	Trained models on PubChem AID 1851 and Veith qHTS panels and fed their predictions into the stack.	Ranked my labels at Spearman 0.28, 0.34, 0.13, and 0.48 (my own models were at 0.58, 0.71, 0.43, and 0.81). On CYP2D6, a model trained on a different isoform was the best public predictor.
Bigger pretraining table	Pretrained an encoder on ~150,000 molecules across 667 columns (ToxCast, MUV, HIV, Tox21, PubChem, ChEMBL).	Lost to an encoder trained on 33,000 molecules across 20 columns. The binary toxicity assays watered down the potency signal.
Tox21 CYP1A2 model	Stripped 864 luciferase counter-screen hits (AID 1224835) out of AID 1671199 and trained GBDT on 4,995 rows.	Got 0.411 Spearman on challenge CYP1A2 labels vs. 0.564 for my existing arm; holdout Spearman barely moved (0.5884 to 0.5905).
Training on the interval metric	Refit models with majorize-minimize on the clipped credible interval before linear calibration.	Helped before calibration (0.7442 to 0.7244), then did worse after calibration (0.7198 vs. 0.7060 for normal loss). Both push guesses toward the interval edges, so they don't stack.
Pairwise delta models	Predicted potency differences from the closest training neighbor.	Scored 0.4488 and 0.5426 vs. 0.5881 and 0.7060 for normal models. Test rows are close to train rows (0.578), but train rows are far apart from each other (0.25 floor).
5 multitask net variants	Added max-response heads, TDI-arm pIC_{50} heads, and swapped encoders against `mtl13` (0.6262 dev).	None beat my +0.015 dev cutoff. The TDI heads got 0.6245 dev and 0.6490 holdout (vs. 0.6652 control).
10 other molecular encoders	Tested MiniMol, Uni-Mol, ChemBERTa, MoLFormer, and 6 others inside TabPFN.	My original encoder beat all of them (0.6370 dev vs. 0.6184 for MiniMol and 0.5911 for Uni-Mol). Chemical LLMs only added +0.0014 and +0.0011.

Idea	What I ran	What happened
Descriptor baselines & Mitra	Tuned LightGBM, RF, SVM, and Ridge on descriptors; tried the Mitra tabular foundation model.	Tree and linear baselines trailed the blend by 0.05–0.08. Mitra ran out of T4 VRAM and had to drop down to 256 examples.

4. How the Final Submission Was Built

4.1 My five arms and the L_1 combiner

My standalone regression file (v46, which scored 0.4472) blends five models trained on 4,905 compounds across five Bemis-Murcko scaffold folds:

1. **TabPFN + `surrogate_adme` (with 2D6 blanks):** TabPFN regressor on 605 raw columns (RDKit 2D descriptors, ECFP4 counts, and the organizers' `surrogate_adme` embedding) with the blank-cell imputation on CYP2D6.
2. **TabPFN + Chemprop embedding:** Same 605-column setup, swapping in a frozen pretrained Chemprop embedding.
3. **Primary-screen LightGBM:** Out-of-fold LightGBM predictions trained on the single-concentration screen.
4. **NB33 (CheMeleon fine-tune):** Pretrained CheMeleon weights ($d_h = 2048$, depth 4) fine-tuned with an interval hinge loss on the four main heads and MAE on 18 extra heads (single-concentration $\log_2$ fold change, ChEMBL pIC_{50}, and PubChem qHTS AC_{50} and max response), following Scigantic's public 22-column recipe on my folds. I had a rule that a new model needed to beat 4 out of 5 folds and improve dev by +0.005: a single NB33 seed only gained 0.0022 and failed, while averaging three seeds (1042, 2042, 3042) gained 0.0062 and passed. Adding NB33 dropped my CYP1A2 board score from 0.5184 to 0.4932 in v38.
5. **NB38 (random-init inverse-variance net):** To figure out what my stack was missing, I took Scigantic's 13 published model columns and tested adding them one by one to my combiner on held-out folds. Their pretrained models didn't help my blend, but their randomly initialized, inverse-variance-weighted net (arm B) gained 0.019 on CYP2C9 and 0.017 on CYP2D6. So I built NB38 the same way: I took my NB33 setup, stripped out the CheMeleon starting weights, trained on MAE across all 22 heads, and weighted each molecule by the mean inverse variance of its credible intervals (normalized to the median and clipped to $[0.25, 4.0]$, which clips 37% of the 4,179 rows). Training five folds took about five hours on a GPU. NB38 ended up getting the biggest weight in my combiner on CYP2C9 and CYP2D6 and the second biggest on CYP1A2, giving me about a third of my standalone board gain.

To blend all ~50 saved prediction columns from my runs, I set up an L_1-penalized linear program for each isoform that minimizes the ST-RAE hinge error on out-of-fold rows. That combiner took my standalone stack from 0.4549 down to 0.4472 (cutting 0.0263 off CYP1A2).

For the TDI classification track (`src/tdi.py`, scored on MCC for CYP3A4 and CYP2D6), I combined multi-task neural shift predictions and primary-screen features inside LightGBM classifiers, then tuned the probability cutoffs on out-of-fold MCC for the 2-fold IC_{50} shift rule (including inferred positives where the TDI-arm $pIC_{50} > 4.301$ and direct $pIC_{50} < 4$).

4.2 Averaging with Scigantic's R25

For my final regression uploads, I averaged my v46 stack with Scigantic's open-source R25 submission (`github.com/Scigantic/open-admet-challenge-cyp`, MIT-0 license). I gave R25 a 0.500 weight on CYP1A2, CYP2D6, and CYP3A4, and 0.400 on CYP2C9, so 47.5% of the final numbers come from their file.

Isoform	v46 (my stack)	R25 (Scigantic)	v47 (blend)
CYP1A2	0.4669	0.4739	0.4545
CYP2C9	0.3609	0.3802	0.3585
CYP2D6	0.5669*	0.5442	0.5449
CYP3A4	0.3940	0.3762	0.3760

*(*An earlier CYP2D6 column of mine in v42 scored 0.5316.)*

I uploaded both files on 29 October (v46 scored 0.4472 thirty-eight minutes after the v47 blend scored 0.4335), so R25 lowered the macro error by 0.0137. The blend beat both parent files on CYP1A2, CYP2C9, and CYP3A4. Looking at the other leaderboard metrics, my standalone v46 stack tied the blend on ranking (Spearman 0.7659 vs. 0.7635; Kendall 0.5868 vs. 0.5865) and lost on error size (MAE 0.6752 vs. 0.6547; R^2 0.5842 vs. 0.6033). Basically, my stack orders the test molecules just as well, while R25 puts its numbers closer to the middle of the credible intervals.

On CYP2C9, my v46 column beat R25 by itself (0.3609 vs. 0.3802). In Scigantic's write-up, they mentioned a bug in their R24 step where their Powell combiner fit weights against an R08 baseline out-of-fold, but applied those weights to R17's CYP2C9 column (from R10) on the test set. Giving my own column a 0.600 weight on CYP2C9 took advantage of that gap.

4.3 Shifting the CYP2D6 floor and widening the spread

At the end of every step in Scigantic's combiner, they run a `restore()` function that resets each column's mean and standard deviation back to their R17 baseline. That kept their ensemble from shrinking the prediction spread, and it also meant the overall center and width weren't tuned directly on the board.

In the training set, weak CYP2D6 molecules ($\text{pIC}_{50} < 3.6$) have a median upper credible bound of 3.75. My raw CYP2D6 predictions had a median of 3.20, with 65% of the column below 3.5. Since my dev folds don't have labels for true non-inhibitors, the only way to test weak-compound placement was on the live board one column at a time. Pulling CYP2D6 predictions below 3.5 up toward 3.5 with $x \mapsto 3.5 - g(3.5 - x)$ scored 0.5524 at $g = 0.5$, 0.5556 at $g = 0.3$, and 0.5714 at $g = 0.15$. When I tried putting a flat floor at 3.6 across all four isoforms at once (v39), it hurt every tab.

Between 29 October (0.4335) and my last upload (v56, 0.4192), the rest of the drop came from stretching each column's spread out around its mean. That linear stretch keeps the column mean and every molecule's rank order locked. About 39% of that drop (and roughly half of my total calibration gain across the whole competition) comes from a simple 1.2× stretch on all four columns, which makes up for how ensembling pulls predictions in toward the average.

5. Where Local Validation Misled Me

I kept a locked holdout of 726 scaffold-disjoint molecules and scored each candidate model on it once. Across three early uploads, the difference between my holdout ST-RAE and the leaderboard ST-RAE was −0.0834, −0.0728, and −0.0833. Two of those were within 0.0001 of each other, and using that offset predicted my third upload within 0.0004.

Still, the holdout burned me in two ways. First, one candidate beat my holdout by 0.0046 (with a 90% bootstrap confidence of being better) and then lost by 0.0060 on the leaderboard. A 726-row holdout is too noisy to tell a 0.005 difference apart from luck, even though I had written my early filter gates assuming it could.

Second, on CYP2D6, my holdout and the leaderboard went in opposite directions four separate times: * Adding the blank-cell pseudo-label arm hurt my holdout by 0.019, then gained 0.053 on the board. * Raising that arm's weight hurt the holdout by 0.021, then gained 0.025 on the board. * NB33 gained 0.0172 on the holdout, then lost 0.007 on the board. * My combiner predicted 0.5282 for the blend on CYP2D6, and the board came back at 0.5449.

Since the 750 test molecules were picked by expanding hits on 1A2, 2C9, and 3A4, the blind CYP2D6 test set is mostly inactive compounds. My labelled holdout only has active hits. Whenever I changed how the model handled weak molecules, the hit-only holdout was grading the wrong population.

6. Limits and Credits

Without Scigantic's R25 file, my v46 stack (0.4472) finishes outside the top 10% on the live leaderboard. Scigantic's own best live score on 28 October was 0.4283 (where they added +0.170 to CYP1A2 and +0.140 to CYP2C9 on top of R25). My final v56 upload (0.4192) is 0.0091 ahead of that score, and that margin comes from post-blend spread stretching, not better models. The top teams on the board run six model families compared to my three, and pretrain on much larger bioactivity tables. Also, all of these numbers come from a single dataset of 4,905 training molecules and 750 blind test molecules, where the public leaderboard only scores half the test set and has a single-column noise floor around 0.002.

Part of the pipeline	Who built it	Effect
5-arm stack on 4,905 training rows	GiantWizard	0.4472 standalone
Using blank screen cells as non-inhibitors on CYP2D6	GiantWizard	Rank 55 to 41
CYP2D6 lower-tail compression test	GiantWizard	0.5524 on CYP2D6
CYP2C9 standalone column	GiantWizard	Beat R25 (0.3609 vs. 0.3802)
L_1 linear program combiner (~50 columns)	GiantWizard	0.4549 to 0.4472
Single-column board testing (holding 3 columns fixed)	GiantWizard	4 tests per upload
1-upload moment solver & TDI confusion-matrix math	GiantWizard	Blind set distribution bounds
R25 predictions (0.5 weight on 1A2/2D6/3A4, 0.4 on 2C9)	Scigantic (MIT-0)	−0.0137 macro error
22-column training recipe & NB33/NB38 architecture spec	Scigantic (MIT-0)	Used in Arms 4 & 5

Part of the pipeline	Who built it	Effect
Murphy R^2 calibration & leaderboard moment probing	briford (SuperCowPowers)	Centering setup
Interval regression loss	Drouin et al. (2017)	Clipped loss function
Chemprop, TabPFN, and CheMeleon weights	Original authors	Base models